

Trigger Code: SavedViews Audit Logging

SQL

```
-- SavedViewLogs intentionally does not have a foreign key to SavedViews.  
-- This preserves the audit record after a saved view is deleted.
```

```
CREATE TABLE IF NOT EXISTS SavedViewLogs (  
    logId BIGINT NOT NULL AUTO_INCREMENT,  
    viewId INT NOT NULL,  
    userId INT NOT NULL,  
    actionType ENUM('CREATE', 'UPDATE', 'DELETE') NOT NULL,  
    actionTime TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    message VARCHAR(255) NOT NULL,  
    PRIMARY KEY (logId),  
    INDEX idx_saved_view_logs_user_time (userId, actionTime),  
    INDEX idx_saved_view_logs_view (viewId)  
);
```

```
DROP TRIGGER IF EXISTS trg_saved_views_after_insert;  
DROP TRIGGER IF EXISTS trg_saved_views_after_update;  
DROP TRIGGER IF EXISTS trg_saved_views_after_delete;
```

```
DELIMITER //
```

```
CREATE TRIGGER trg_saved_views_after_insert
```

```
AFTER INSERT ON SavedViews
FOR EACH ROW
BEGIN
    IF NEW.title IS NOT NULL AND CHAR_LENGTH(TRIM(NEW.title)) > 0 THEN
        INSERT INTO SavedViewLogs (viewId, userId, actionType, message)
        VALUES (
            NEW.viewId,
            NEW.userId,
            'CREATE',
            CONCAT('Saved view created: ', LEFT(NEW.title, 220))
        );
    END IF;
END//
```

```
CREATE TRIGGER trg_saved_views_after_update
AFTER UPDATE ON SavedViews
FOR EACH ROW
BEGIN
    IF NOT (OLD.title <=> NEW.title) OR NOT (OLD.filters <=> NEW.filters)
    THEN
        INSERT INTO SavedViewLogs (viewId, userId, actionType, message)
        VALUES (
            NEW.viewId,
            NEW.userId,
            'UPDATE',
```

```

        CONCAT('Saved view updated: ', LEFT(COALESCE(NEW.title,
'(untitled)'), 220))

    );

END IF;

END//

CREATE TRIGGER trg_saved_views_after_delete

AFTER DELETE ON SavedViews

FOR EACH ROW

BEGIN

    IF OLD.viewId IS NOT NULL THEN

        INSERT INTO SavedViewLogs (viewId, userId, actionType, message)

        VALUES (

            OLD.viewId,

            OLD.userId,

            'DELETE',

            CONCAT('Saved view deleted: ', LEFT(COALESCE(OLD.title,
'(untitled)'), 220))

        );

    END IF;

END//

DELIMITER ;

```

This trigger system automatically records CREATE, UPDATE, and DELETE actions on SavedViews in the SavedViewLogs table. The frontend only reads these audit records through the backend; the log rows are generated by MySQL triggers.

Stored Procedure: GetCropEfficiencyReport

-- Generates a multi-year crop yield report categorized by efficiency tiers.

```
DROP PROCEDURE IF EXISTS GetCropEfficiencyReport;
```

```
DELIMITER $$
```

```
CREATE PROCEDURE GetCropEfficiencyReport(
```

```
    IN p_year_start INT,
```

```
    IN p_year_end INT,
```

```
    IN p_min_yield DOUBLE
```

```
)
```

```
BEGIN
```

```
    IF p_year_start > p_year_end THEN
```

```
        SET p_year_start = 1990;
```

```
        SET p_year_end = 2024;
```

```
    END IF;
```

```
    IF p_min_yield IS NULL OR p_min_yield < 0 THEN
```

```
        SET p_min_yield = 1000.0;
```

```
    END IF;
```

```
    SELECT
```

```
        cr.cropId,
```

```
        cr.cropName,
```

```
        COUNT(DISTINCT pr.countryId) AS reportingCountries,
```

```
        ROUND(AVG(pr.value), 2) AS avgYieldKgPerHa,
```

```
        ROUND(MAX(pr.value), 2) AS peakYieldKgPerHa,
```

```
        MIN(pr.year) AS earliestYear,
```

```

        MAX(pr.year) AS latestYear,

        CASE

            WHEN AVG(pr.value) >= p_min_yield THEN 'High Efficiency'

            WHEN AVG(pr.value) >= (p_min_yield * 0.5) THEN 'Moderate
Efficiency'

            ELSE 'Developing Efficiency'

        END AS efficiencyClassification

    FROM ProductionRecords pr

    JOIN Crops cr ON pr.cropId = cr.cropId

    WHERE pr.element = 'Yield (kg/ha)'

        AND pr.year BETWEEN p_year_start AND p_year_end

    GROUP BY cr.cropId, cr.cropName

    ORDER BY avgYieldKgPerHa DESC;

END$$

DELIMITER ;

```

Trigger Code: trg_validate_saved_view_title

```

-- Automatically formats default or missing preset titles before insertion.

DROP TRIGGER IF EXISTS trg_validate_saved_view_title;

DELIMITER $$

CREATE TRIGGER trg_validate_saved_view_title

BEFORE INSERT ON SavedViews

FOR EACH ROW

BEGIN

```

```

        IF NEW.title IS NULL OR TRIM(NEW.title) = '' OR TRIM(NEW.title) =
'Untitled' THEN

            SET NEW.title = CONCAT('Exploratory Preset - ', DATE_FORMAT(NOW(),
'%Y-%m-%d %H:%i'));

        END IF;

END$$

DELIMITER ;

```

Transaction benchmark snapshot

-- Executed inside the backend endpoint (`POST /api/analytics/snapshot`) using
-- REPEATABLE READ isolation to compute regional benchmarks and record a
snapshot.

```

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

START TRANSACTION;

SELECT

    pr.element,

    ROUND(AVG(pr.value), 2) AS benchmarkAverage,

    COUNT(DISTINCT pr.countryId) AS totalCountries,

    COUNT(DISTINCT pr.cropId) AS totalCrops

FROM ProductionRecords pr

JOIN Crops cr ON pr.cropId = cr.cropId

WHERE pr.year BETWEEN 2015 AND 2024

GROUP BY pr.element;

```

```

INSERT INTO SavedViews (userId, title, filters, createdAt)
VALUES (
    1,
    CONCAT('Benchmark Snapshot - ', DATE_FORMAT(NOW(), '%Y-%m-%d')),
    '{"element": "All Metrics", "benchmark": true, "period": "2015-2024"}',
    NOW()
);

COMMIT;

```

Stored Procedure: CreateDefaultSavedView

-- Creates a default regional production preset for each newly registered user.

```

CREATE PROCEDURE CreateDefaultSavedView(IN p_user_id INT)
INSERT INTO SavedViews (userId, title, filters)
SELECT
    p_user_id,
    'Regional Production Overview',
    JSON_OBJECT(
        'element', 'Production (tonnes)',
        'cropIds',
        (
            SELECT JSON_ARRAYAGG(x.cropId)
            FROM (
                SELECT pr.cropId

```

```

FROM ProductionRecords pr

JOIN Crops cr ON pr.cropId = cr.cropId

WHERE pr.element = 'Production (tonnes)'

      AND pr.unit = 't'

      AND pr.year BETWEEN 2020 AND 2024

      AND LOWER(cr.cropName) NOT LIKE '%primary%'

GROUP BY pr.cropId, cr.cropName

ORDER BY SUM(pr.value) DESC

LIMIT 3

    ) x

),

'countryIds',

(

    SELECT JSON_ARRAYAGG(countryId)

    FROM Countries

),

'yearRange',

JSON_ARRAY(2020, 2024)

);

```


Trigger Code: New User Default Saved View

-- Calls the stored procedure after a new user account is created.

DROP TRIGGER IF EXISTS trg_new_user_saved_view;

CREATE TRIGGER trg_new_user_saved_view

AFTER INSERT ON Users

FOR EACH ROW

CALL CreateDefaultSavedView(NEW.userId);